Знание вне промта

Процессы на больших языковых моделях: четвёртое занятие дуги.

Что остаётся от прошлого занятия

Процесс, с которым вы ушли с прошлого занятия, умел пять вещей: отвечать обратившемуся вместо того, чтобы молчать; проверять собственный ответ перед отправкой; решать по сумме возврата правилом, записанным в коде, вместо угадывания моделью; отвечать в одном устойчивом формате на каждый вызов; и, если к заявке приложена фотография, читать её вместо того, чтобы просто сохранять как вложение.

Вопрос, которым закончился С9, дословно: «При каком размере и изменчивости базы знаний „всё в промт“ перестаёт быть разумным?»

Учебный центр Ольги не стоит на месте: к базе каждую неделю что-то добавляется — новый регламент, обновлённый FAQ, разбор очередного спорного случая. Десять пунктов С9 — это база на момент запуска процесса; дальше она растёт.

Ось занятия: откуда процесс берёт факты и почему считает их верными

ВЫГЛЯДИТ НАДЁЖНЫМ	ЧЕМ ОКАЗЫВАЕТСЯ
если база выросла — просто перечислим её всю в промте, как на С9	
порезать документ на части можно как угодно, главное чтобы кусок попал в хранилище	
поиск по смыслу (вектор) и так находит то же самое, что поиск по словам, только умнее	
если умеем искать текст — картинку найдём тем же способом, просто подписав её словами	

Откуда процесс берёт факты и почему считает их верными?

Четыре строки — четыре кейса занятия. Правая колонка заполняется измеренным: прогоном нашего собственного стенда на реальных данных.

Четыре кейса и один разбавитель

1. **Хранение базы знаний** — что происходит с точностью и стоимостью ответа, когда база растёт с десяти пунктов до сотен документов.
2. **Разбиение документа на части** — когда база знаний превращается из десяти строчек в настоящие документы, их нужно сначала разрезать; от места разреза зависит, останется ли ответ на вопрос целым.
3. **Чем искать нужный кусок** — кусков в базе становится много, и способ поиска (по словам или по смыслу) не взаимозаменяем.
4. **Поиск по изображениям** — та же задача хранения и поиска, только для сканов и фотографий, которые учебный центр получает в заявках.

Разбавитель: система способна найти топически релевантный фрагмент и всё равно дать неверный ответ. «Найдено» и «верно» — разные вещи.

Кейс 1. Хранение базы знаний

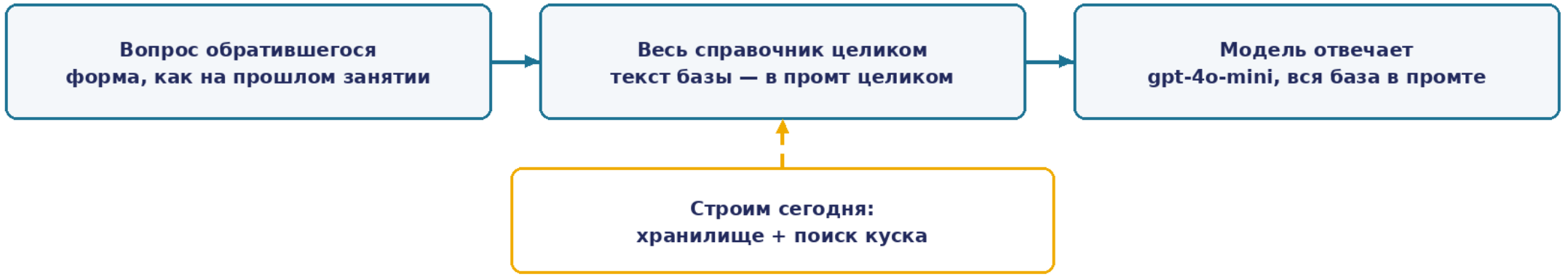
Справочник учебного центра растёт каждую неделю: новый регламент, обновлённый пункт FAQ, разбор очередного спорного случая. Десять пунктов C9 — стартовая точка; дальше справочник только растёт.

Хранение базы знаний | Разбиение документа на части | Чем искать нужный кусок | Поиск по изображениям

Что Ольге нужно от растущей базы

Где в потоке ответа встанет поиск нужного куска

форма и вызов модели уже работают — вопрос в том, что займёт место узла «весь справочник целиком», когда база вырастет



Потребность. Ответ остаётся точным и дешёвым, даже когда база вырастет с десяти пунктов до сотен документов.

1. Версия 1 — естественное продолжение С9: дописывать базу в тот же промт, без ограничения сверху.
2. Ещё два реальных способа: искать по совпадению слов; искать по смыслу — через векторное хранилище.

Все три — реальные инженерные варианты. Какой выдержит рост базы — вопрос следующего экрана.

Как устроить базу знаний, которая растёт

Как устроить базу знаний, которая растёт?

продолжать дописывать базу в тот же промт

искать по ключевым словам

положить в векторное хранилище и вытаскивать нужный кусок

Что делает каждое из трёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Продолжать дописывать базу в тот же промт	Весь текст базы на каждый вопрос заново уходит модели целиком	Не требует вообще никакой подготовки заранее — база просто лежит текстом в настройках процесса	Чем больше база, тем больше текста уходит на каждый вопрос — расход растёт вместе с базой
Искать по ключевым словам	Пересечение слов запроса и текста документа, без единого обращения к модели	Дёшево и быстро независимо от размера базы; подготовка — индексация по словам — делается один раз до прихода первого вопроса	Находит совпадение слов: смысл при этом не учитывается — вопрос, заданный другими словами, может не найти кусок, где ответ на самом деле есть
Положить в векторное хранилище и вытаскивать нужный кусок	База заранее превращается в числовые представления смысла; по запросу находится то, что ближе всего по смыслу, без точного совпадения слов	В промт попадает только найденный кусок — объём всей базы на расход не влияет	Требует отдельного шага подготовки заранее: базу нужно разбить на куски и посчитать для каждого эмбеддинг, прежде чем придёт первый вопрос

Структура замера: десять вопросов, четыре размера, три действия

Модель и настройки. `gpt-4o-mini` — ответ по переданному тексту; `text-embedding-3-small` — эмбеддинг для поиска. Оба — без явно заданных дополнительных настроек, один и тот же вызов на всех двенадцати комбинациях.

ЧТО ВАРЬИРУЕТСЯ	ЗНАЧЕНИЯ
Размер базы	10 · 50 · 200 · 500 документов
Действие	промт целиком · поиск по словам → ответ · поиск по смыслу → ответ
Вопросы	один и тот же набор из 10, одинаковый на всех размерах и действиях

Пример. «Как называется круг кровообращения, функционирующий через лёгкие?» — правильный ответ: «малый» (документ `needle-1`, присутствует в базе на всех четырёх размерах).

Что показал прогон на четырёх размерах базы

ACCURACY	ПРОМТ ЦЕЛИКОМ	KEYWORD → ОТВЕТ	VECTOR → ОТВЕТ
10 докум.	0,7	0,5	0,6
50 докум.	0,7	0,5	0,7
200 докум.	0,7	0,4	0,6
500 докум.	**ОШИБКА HTTP 400**	0,5	0,7

$/ВОПРОС	ПРОМТ ЦЕЛИКОМ	KEYWORD → ОТВЕТ	VECTOR → ОТВЕТ
10 докум.	$0,000189	$0,0000484	$0,0000486
50 докум.	$0,000809	$0,0000484	$0,0000482
200 докум.	$0,003204	$0,0000501	$0,0000481
500 докум.	— отклонён	$0,00005	$0,000049

Retrieval recall@1: у keyword — **0,8 на всех четырёх размерах без изменения**; у vector — **1,0 на 10/50/200, 0,9 на 500**. **На 500 документах промт-стаффинг требует около 160 000 токенов — больше контекста gpt-4o-mini (128 000); OpenRouter отклоняет запрос с HTTP 400 ещё до ответа модели.**

Две систематические ошибки поиска, разной природы

Ошибка 1 — keyword, падежные формы (на всех размерах базы одинаково)

ВОПРОС	ПЕРЕСЕЧЕНИЕ С ВЕРНЫМ ДОКУМЕНТОМ	ПЕРЕСЕЧЕНИЕ С НАЙДЕННЫМ (НЕВЕРНЫМ)	ВЕРДИКТ
q4: «...палеонтологами Нильсов Элдриджем и Стивеном Гулдом...» → эталон «1972»	needle-4: «...в 1972 году палеонтологи Элдридж и Гулд...» — 1 слово («году»)	needle-2 (про Моцарта): 1 слово («году») — ничья, взят не тот	**НЕВЕРНО**
q10: «...модель системы мира... взята за основу Кеплером?» → эталон «Коперник»	needle-10: «...кеплеровская система мира основана на модели Коперника...» — 1 слово («мира»)	needle-3 (про марафон): 2 слова («модель», «мира») — перевес не в пользу верного	**НЕВЕРНО**

Падежные формы keyword не считает совпадением — ошибка держится и на 10 документах без дистракторов: причина не в масштабе базы, она в методе.

Ошибка 2 — vector, соседний абзац той же статьи (только на 500 документах)

ВОПРОС	НАЙДЕННЫЙ ДОКУМЕНТ	ОТВЕТ МОДЕЛИ	ВЕРДИКТ
q3: «Какая температура летом не даёт марафонцам показывать наилучшие результаты?» → эталон «высокая», документ needle-3	`distractor-237` вместо needle-3 — вероятно, соседний абзац той же статьи (общая сноска [17]) с буквальным числом	«Температура выше 14—16 °C ухудшает результаты марафонцев.»	**НЕВЕРНО** (по эталону — содержательно разумно)

Keyword здесь нашёл ПРАВИЛЬНЫЙ needle-3 — но модель ответила «нет информации»: числа в нём не было, оно лежало в абзаце, который достался vector-варианту.

Что подтвердил замер по каждому из трёх действий

Модель, которая формирует финальный ответ во всех трёх действиях, — одна и та же: `gpt-4o-mini`, без явно заданных настроек вызова. Действие 3 дополнительно использует `text-embedding-3-small` для поиска.

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Дописывать всё в промт	Весь текст базы на каждый вопрос заново уходит модели целиком	Accuracy держится на одном уровне, 0,7, на 10/50/200 документах — без видимого предупреждающего сигнала	Стоимость растёт в 17,0 раза на 10→200 документах при той же точности; на 500 документах не работает вовсе — запрос превышает контекст модели, без предварительного падения качества
Искать по ключевым словам	Пересечение слов запроса и документа, без модели	Дёшево и быстро независимо от размера базы (~$0,00005)	Retrieval recall@1 = 0,8 на всех четырёх размерах без изменения: держится ровно на месте независимо от объёма базы — причина в сравнении слов без учёта падежных форм (разбор — отдельным тактом)
Класть в векторное хранилище	Эмбеддинг запроса сравнивается с эмбеддингами кусков базы, в промт идёт только найденный кусок	Промт держит ~270 токенов и ~$0,00005 независимо от размера базы; retrieval recall@1 = 1,0 на 10/50/200 документах	Требует отдельного шага подготовки (разбить базу, посчитать эмбеддинги) до того, как придёт первый вопрос; recall@1 проседает до 0,9 на 500 документах — на таком масштабе соседние абзацы одной статьи начинают путать поиск по смыслу (разбор — отдельным тактом)

Где хранится база и что видит модель

Вместо того чтобы на каждый вопрос передавать модели всю базу целиком, процесс хранит базу отдельно от разговора. По каждому новому запросу он сначала находит в этом отдельном хранилище несколько подходящих кусков и только их кладёт в промт, оставляя остальную базу в стороне.

Настоящий холст: векторное хранилище в n8n



И шаг эмбеддинга, и финальный вызов модели собраны обычным узлом HTTP Request с предустановленным типом учётных данных — отдельного штатного узла «эмбеддинг через OpenRouter» в каталоге n8n нет.

Настоящая панель: код узла поиска

JavaScript

```
const payload = JSON.parse($('Вопрос + корпус с векторами').first().json.payload);
const qvec = $input.first().json.data[0].embedding;
function cos(a,b){let d=0,na=0,nb=0;for(let i=0;i<a.length;i++){d+=a[i]*b[i];na+=a[i]*a[i];nb+=b[i]*b[i];}return d/(Math.sqrt(na)*Math.sqrt(nb));}
let best=null,bestScore=-1;
for(const doc of payload.corpus){const s=cos(qvec,doc.embedding);if(s>bestScore){bestScore=s;best=doc;}}
return [{json:{question:payload.question,chunk:best.text,doc_id:best.id,score:bestScore,model:payload.model}}];
```

Узел «Simple Vector Store» — это цикл по всем документам корпуса и ручная функция косинусного сходства: без вызова отдельного сервиса.

Настоящая панель: промт узла ответа

Prompt (User Message)

```
Отвечай ТОЛЬКО по тексту ниже. Если ответа нет в тексте — напиши «нет информации».

Текст:
{{ $json.chunk }}
```

> *«Отвечай ТОЛЬКО по тексту ниже. Если ответа нет в тексте — напиши «нет информации». Текст: `{{ $json.chunk }}`»*

Что видно Ольге, а что — обратившемуся

Обратившемуся это не видно вовсе — он получает тот же по качеству или лучший ответ. Ольге видно другое: при текущем объёме базы (пока он мал) разница почти не заметна, но именно от неё зависит, во сколько обходится каждый следующий ответ, когда база вырастет.

Принцип переносится на любой оркестратор: n8n здесь просто пример. Промт, в который целиком кладётся база знаний, растёт в стоимости линейно с размером базы и не гарантирует роста точности — внешнее хранилище, из которого на каждый вопрос достаётся только нужный кусок, держит стоимость и объём промта постоянными независимо от того, сколько документов накопилось.

Когда хранилище окупается, а когда — нет

Критерий. Внешнее хранилище окупается, когда рост базы делает промт-стаффинг либо заметно дороже, либо структурно невозможным — а не самим фактом наличия базы.

КОГДА НУЖНО

- База растёт заметно быстрее, чем раз в год, и счёт идёт на сотни документов
- Стоимость вызова уже измеримо больше нуля и видна на счёте за месяц
- Размер базы может однажды превысить контекст модели

КОГДА НЕ НУЖНО

- База — те же десять-двадцать пунктов FAQ, что и на прошлом занятии, и расти не планирует
- Разница по стоимости ($0,0002 против $0,00005) ещё не оправдывает отдельный шаг подготовки
- До предела контекста далеко, и структурного отказа не предвидится

Устаревший индекс — типовая проблема ступени, и эта находка касается не только этого прогона. Векторное хранилище — копия базы на момент, когда её туда положили: живой ссылки на источник оно не держит. Если регламент в исходном документе поменялся, а переиндексация ещё не прошла, хранилище честно находит и отдаёт кусок — только устаревший.

Первая строка оси занятия закрыта: современная модель от роста базы не ломается структурно — она дорожает монотонно до определённого объёма, а затем перестаёт отвечать вовсе; внешнее хранилище держит стоимость и recall@1 = 1,0 постоянными на любом из проверенных размеров.

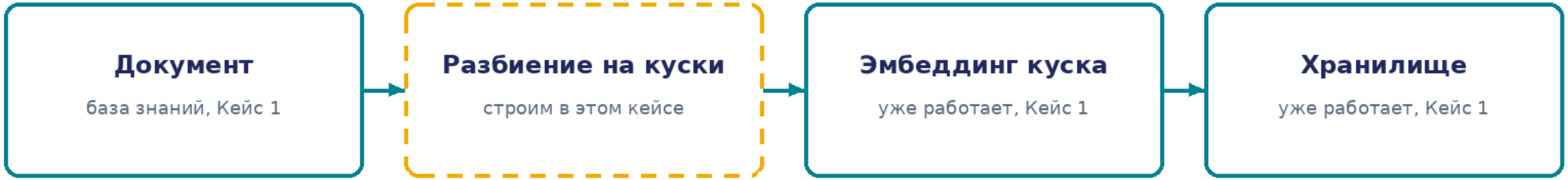
Кейс 2. Разбиение документа на части

Кейс 1 решил, где база будет жить, когда вырастет — в отдельном хранилище, вне промта. Но хранилище не глотает документ целиком: трёхстраничный регламент не укладывается в один кусок так, чтобы поиск потом мог точно указать на нужное место.

Как резать документ, чтобы кусок не терял смысл

Где в пути документа встаёт разбиение на куски

эмбеддинг и хранилище уже работают с прошлого занятия — вопрос в том, что происходит до них



Потребность. Каждый кусок хранилища несёт законченный кусок смысла — ответ на вопрос не должен оказаться разорван между двумя соседними кусками.

Версия 1 — естественная первая попытка: резать документ на части фиксированного размера, по счёту символов, безотносительно к границам предложений.

Один параметр, один проход по тексту. Сработает ли это для смысла — проверяем дальше.

Где должен проходить разрез документа

Где должен проходить разрез документа?

- **резать строго по счёту символов, без оглядки на текст**
- **резать по границам предложений**
- **резать по границам слов**
- **резать по счёту символов с перекрытием**

Что делает каждое из четырёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Резать строго по счёту символов, без оглядки на текст	Документ рубится на части заданной длины без оглядки на текст внутри	Проще всего реализовать — один параметр, один механический проход по тексту	Разрез может прийтись ровно на середину фразы с ответом, независимо от выбранного размера куска; неочевидно заранее, какой размер «удобнее» без проверки
Резать по границам предложений	Разрез проходит только там, где заканчивается одно предложение и начинается следующее	Куски не рвут фразу посередине: разрез идёт по смысловой границе текста — счёт символов тут роли не играет	Чуть сложнее в реализации: нужен отдельный шаг, который понимает, где заканчивается предложение в конкретном языке — простого счёта символов здесь недостаточно; качество этого шага зависит от текста
Резать по границам слов	Документ делится на слова; кусок копится, пока не превысит целевой размер, разрез проходит только на границе между двумя соседними словами	Не рвёт слово пополам; реализовать проще, чем разбор границ предложений — достаточно знать, где кончается слово	Про границу предложения этот способ ничего не знает: разрез всё равно может пройти ровно посередине фразы, даже если ни одно слово не задето
Резать по счёту символов с перекрытием	Та же логика, что резать по счёту символов, но соседние куски перекрываются — часть символов конца одного куска повторяется в начале следующего	Часть ответов, разорванных на границе первого действия, попадает целиком в соседний перекрывающий кусок	Хранилище растёт из-за дублирования символов; перекрытие не гарантирует результат для ответа любой длины

Структура замера: что и на чём проверяли

Корпус. SberQuAD, 60 документов (не короче 500 символов — иначе резать не на что), 72 вопроса с разметкой ответа, проверенной напрямую по тексту.

Находка по пути. У 22,0% строк исходного датасета (1 108 из 5 036) положение ответа не совпадало с текстом документа — такие строки исключены из замера целиком, без подгонки под удобный результат.

Пять групп — один и тот же размер куска для всех четырёх версий: 120 · 180 · 220 · 300 · 450 символов. 180 — заголовочное значение этого занятия.

Пример — реальный прогон демо-стенда: документ про кибернетику, 587 символов. Вопрос: «Из каких источников могут быть потоки информации в определении Гордона Паска?» Эталонный ответ — «из любых источников», позиции 528–547: буквальная подстрока документа на этом месте, её никто не придумывал.

Модели на этом шаге нет. Ни одна из четырёх версий резки не вызывает ни LLM, ни эмбеддинг — все четыре узла «Нарезка» (следующий такт) обычный код. Модель появляется позже, на шаге эмбеддинга уже готового куска (Кейс 1), которого этот замер не трогает.

Результатов на этом кадре нет — что из этого вышло, дальше.

Что показал прогон на пяти размерах куска

72 вопроса, заголовочный размер куска — 180 символов. Версия 1 — 9 разорвано (12,5%), 63 целиком (87,5%). Версия 2 — 1 разорван (1,4%), 71 целиком (98,6%).

На том же размере куска: версия 3 (слова) — 6 разорвано (8,3%); версия 4 (счёт символов с перекрытием) — 4 разорвано (5,6%).

РАЗМЕР КУСКА, СИМВОЛОВ	V1 — СЧЁТ СИМВОЛОВ	V2 — ПО ПРЕДЛОЖЕНИЯМ	V3 — ПО СЛОВАМ	V4 — СЧЁТ СИМВОЛОВ + ПЕРЕКРЫТИЕ
120	9,7% (7 из 72)	1,4% (1 из 72)	9,7% (7 из 72)	5,6% (4 из 72)
180 (заголовочное)	12,5% (9 из 72)	1,4% (1 из 72)	8,3% (6 из 72)	5,6% (4 из 72)
220	4,2% (3 из 72)	1,4% (1 из 72)	1,4% (1 из 72)	1,4% (1 из 72)
300	6,9% (5 из 72)	0,0% (0 из 72)	5,6% (4 из 72)	1,4% (1 из 72)
450	2,8% (2 из 72)	1,4% (1 из 72)	0,0% (0 из 72)	0,0% (0 из 72)

Разбор на настоящем документе: где прошёл разрез

Документ (тот же пример, что в структуре теста): «…Гордон Паск расширил определение, включив потоки информации из любых источников, начиная со звёзд и заканчивая мозгом.» Эталонный ответ — «из любых источников», позиции 528–547.

ВЕРСИЯ	ГДЕ ПРОХОДИТ РАЗРЕЗ РЯДОМ С ОТВЕТОМ	ВЕРДИКТ
Резать по счёту символов (180)	кусок 3 кончается на «…из любых ист», кусок 4 начинается с «очников…» — разрез на 540-м символе прошёл внутри слова	**РАЗОРВАН**
Резать по границам предложений	весь ответ — внутри одного куска [411–587): «…Гордон Паск расширил определение, включив потоки информации из любых источников…»	**ЦЕЛИКОМ**
Резать по границам слов	граница куска стоит ровно на 528-м символе — на самом начале ответа; кусок 4 [528–587) начинается словом «источников» и несёт весь ответ	**ЦЕЛИКОМ**
Резать по счёту символов с перекрытием	перекрытие сдвигает начало последнего куска до 459-го символа; весь ответ (528–547) укладывается внутрь куска [459–587)	**ЦЕЛИКОМ**

Систематическая ошибка версии 1. Резка по счёту символов не смотрит на текст вообще — граница куска может пройти где угодно, в том числе внутри одного слова. Здесь 540-й символ документа оказался ровно в середине слова «источников».

Один документ иллюстрирует механизм; вывод на всю выборку даёт свод. На этом документе версии 3 и 4 сохраняют ответ целиком, но свод по шестидесяти документам (предыдущий такт) показывает их в среднем заметно хуже версии 2 — 8,3% и 5,6% разорванных ответов против 1,4%.

Вывод. Это не единичная неудача одного документа — свод по шестидесяти документам и семидесяти двум вопросам показывает тот же эффект в среднем: версия 1 систематически режет ответы на части, версия 2 — почти никогда.

Что подтвердил замер по каждому из четырёх действий

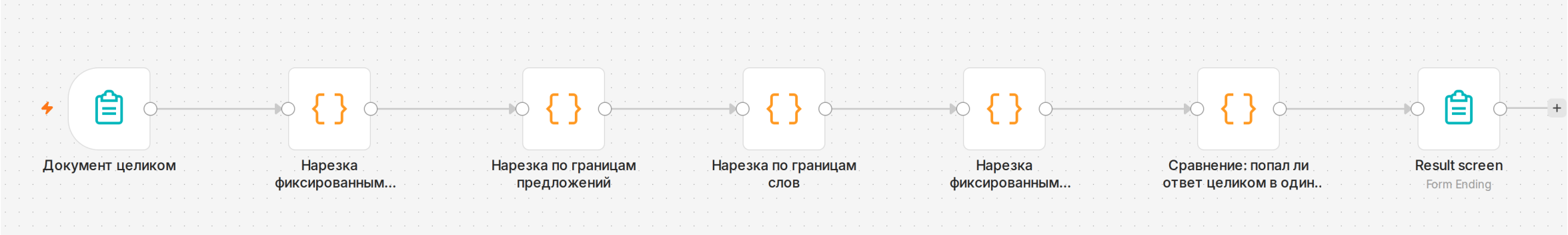
ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Резать по счёту символов	Документ рубится на части заданной длины без оглядки на текст внутри	Проще всего реализовать — один параметр, один проход	Разрывает ответ на вопрос в 2,8–12,5% случаев в зависимости от размера; неочевидно, какой размер «удобнее» без проверки
Резать по границам предложений	Разрез проходит только там, где заканчивается предложение	Доля разорванных ответов падает в разы — почти всегда 0–1,4% на всех проверенных размерах	Чуть сложнее в реализации (нужен разбор текста на предложения — одного счёта символов тут мало); зависит от качества разбивки на предложения в конкретном языке и тексте
Резать по границам слов	Документ делится на слова, кусок копится до целевого размера, разрез только на границе между словами	Дешёвая альтернатива версии 1: держит слово целым и реализуется проще разбора предложений	Разрывает ответ в 0–9,7% случаев: держит границу слова, но не знает о границе предложения
Резать по счёту символов с перекрытием	Та же логика, что версия 1, но соседние куски перекрываются на часть длины куска	Разрывает ответ реже версии 1 — 0–5,6% против 2,8–12,5%	Хранилище растёт от дублирования символов; не закрывает разрыв до уровня версии 2

Где должна проходить граница куска

Граница куска должна совпадать с границей смысла внутри документа — счёт символов тут значения не имеет. Резка, которая режет документ «как придётся», экономит один шаг подготовки и платит за это тем, что часть ответов на будущие вопросы окажется разорвана между двумя кусками.

Это решение не нуждается в модели. Все четыре версии резки — по счёту символов, по границам предложений, по границам слов и по счёту символов с перекрытием — чисто алгоритмические: место границы вычисляется кодом, без обращения к LLM.

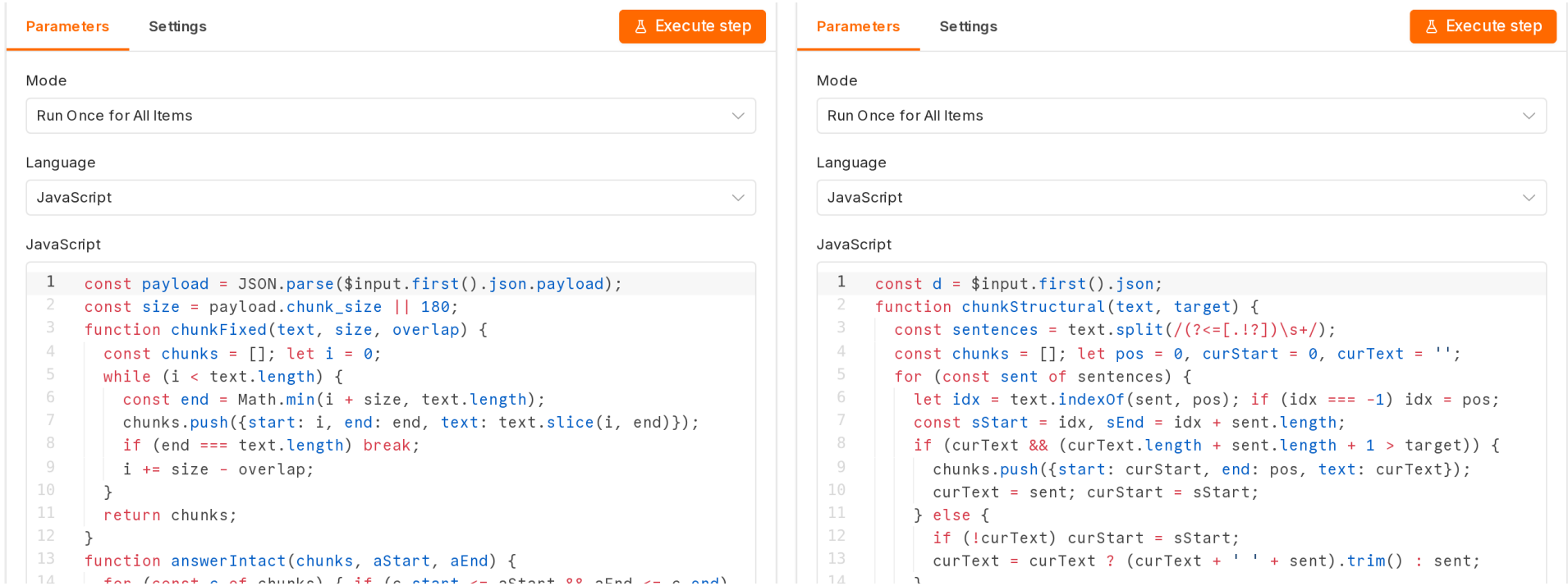
Настоящий холст: сравнение четырёх версий на одном документе



Документ целиком → Нарезка фиксированным размером → Нарезка по границам предложений → Нарезка по границам слов → Нарезка фиксированным размером с перекрытием → Сравнение: попал ли ответ целиком в один кусок → экран результата.

На этом холсте нет шага эмбеддинга и нет записи в хранилище — это отдельный демо-сценарий, который прогоняет все четыре версии нарезки на одном документе и сравнивает их прямо на экране. Шаг эмбеддинга куска (модель `text-embedding-3-small`, Кейс 1) стоит в другом сценарии, после того как кусок уже готов, и не меняется ни одной настройкой от того, какая версия нарезки его подготовила.

Настоящий экран: как устроены два узла нарезки



Нарезка фиксированным размером (слева) читает `payload.chunk_size` (по умолчанию 180), затем в цикле `while (i < text.length)` режет строку кусками по этому числу символов подряд, не глядя на содержимое — после каждого куска сдвигается на `size` символов и повторяет. Нарезка по границам предложений (справа) сначала делит текст на предложения регулярным выражением (граница — после точки, восклицательного или вопросительного знака), затем накапливает предложения в один кусок, пока суммарная длина не превысит `target` символов, — кусок обрывается только на границе предложения, никогда внутри него. Оба узла — обычный код (Code-узел, язык JavaScript, режим «Run Once for All Items»), без единого обращения к модели.

Что видно Ольге, а что — обратившемуся

Обратившемуся результат виден напрямую: если ответ на его вопрос разорван между двумя кусками, модель получает только половину нужного факта и отвечает неполно или неверно: дело тут не в качестве поиска — то, что он нашёл, само по себе было неполным. Ольге видно число — какая доля вопросов рискует получить разорванный ответ при текущей настройке резки.

Принцип общий: применим к любому инструменту резки текста, и узел нарезки в n8n здесь лишь один пример. Кусок хранилища — это единица, внутри которой должен сохраняться смысл; выбор границы разреза прямо определяет, сколько ответов придут целыми, и весит он куда больше, чем обычная техническая настройка.

Что показал более полный замер

Размер куска не ведёт себя монотонно сам по себе. У версии 1 доля разорванных ответов не снижается плавно с ростом размера куска: 9,7% → 12,5% → 4,2% → 6,9% → 2,8%. Выбор «удобного на вид» размера не гарантирует качества — требуется проверка на своих данных: чужое эмпирическое правило тут не перенести.

Перекрытие даёт частичный выигрыш. На заголовочном размере добавленное перекрытие снижает долю разорванных ответов версии 1 почти вдвое — с 12,5% до 5,6%. До уровня версии 2 (1,4%) разрыв не закрывается.

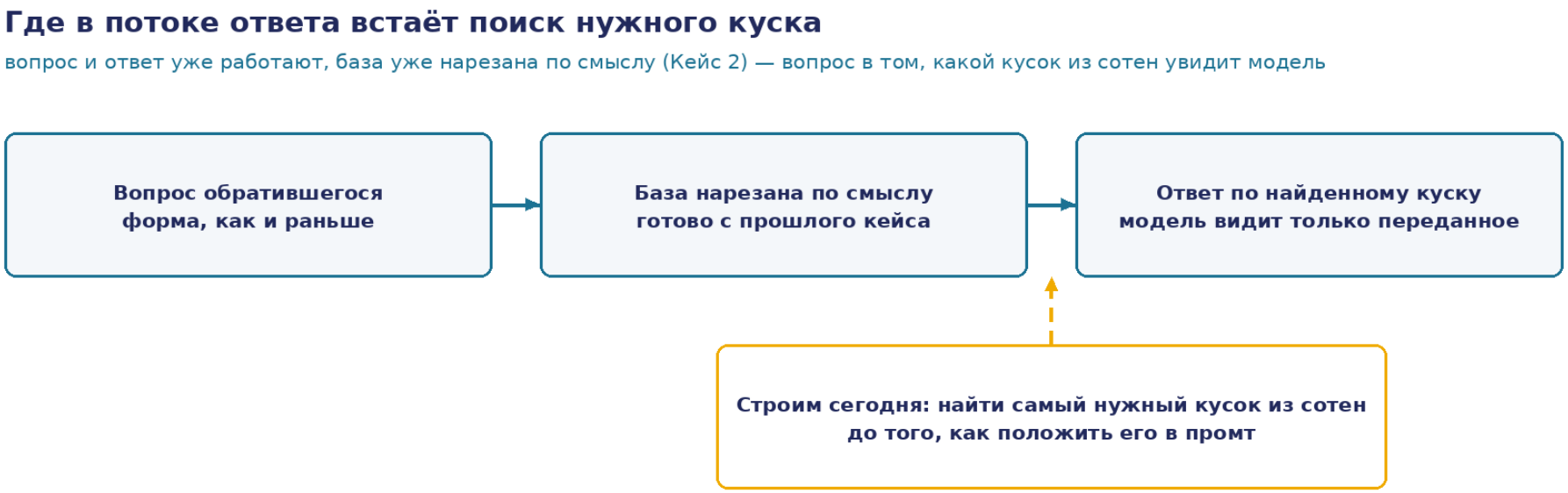
Резка по словам — та же картина. На том же размере доля разорванных ответов снижается с 12,5% до 8,3% — слово защищено от разрыва, предложение всё равно не защищено.

Вторая строка оси занятия закрыта: порезать документ на части МОЖНО как угодно технически — но от этого напрямую зависит, какая доля будущих ответов придёт целой, и дешёвые доработки симптома снижают эту долю, но не убирают её.

Кейс 3. Чем искать нужный кусок

Кейс 2 разрезал документы правильно, по смыслу — счёт символов тут роли не играл. Но правильно нарезанный кусок ещё нужно найти среди сотен таких же, когда приходит конкретный вопрос.

Как найти нужный кусок среди сотен таких же



Потребность. Среди сотен правильно нарезанных кусков базы найти ровно тот, в котором лежит ответ на заданный вопрос — и найти его именно на первом месте выдачи — десятое тут не считается.

Версия 1 — после того, что показали Кейсы 1 и 2 (поиск по смыслу работает, резка по смыслу работает), естественно предположить: искать тоже нужно по смыслу — через векторное сравнение запроса с кусками базы. Кажется, что раз такой поиск понимает смысл вопроса, он заведомо лучше поиска по совпадению слов.

Это и есть интуиция, которую кейс проверяет замером.

Чем искать нужный кусок среди сотен

Чем искать нужный кусок среди сотен?

искать по совпадению слов

искать по смыслу (вектор)

искать и тем, и тем, объединить результаты

отдать финальный порядок на пересмотр модели

Что делает каждое из четырёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Искать по совпадению слов	Совпадение токенов запроса и документа, без обращения к модели	Дёшево и быстро — сам поиск ничего не стоит	Находит совпадение слов — смысл он не распознаёт: лексический разрыв между вопросом и документом, а также похожие по теме, но разные по сути документы сбивают точное совпадение
Искать по смыслу (вектор)	Эмбеддинг запроса сравнивается с эмбеддингами кусков базы — побеждает тот, что ближе по этому числовому представлению	Находит по смыслу: совпадением слов не ограничивается — переформулировка вопроса другими словами не мешает поиску	Слабее там, где важно точное совпадение конкретного имени или числа — смысловая близость не гарантирует точного совпадения детали
Искать и тем, и тем, объединить результаты	Два независимых ранжирования — по словам и по смыслу — сливаются в одно по правилу объединения рангов	Сочетает силу обоих сигналов там, где они расходятся — берёт то, что нашёл хотя бы один из двух способов	Слабый сигнал поиска по словам иногда перетягивает объединённый результат туда, куда чистый поиск по смыслу сам бы не пошёл
Отдать финальный порядок на пересмотр модели	После того как список кандидатов уже собран одним или несколькими способами поиска, отдельный вызов модели читает и вопрос, и всех кандидатов целиком, и пересматривает их порядок	Модель видит кандидатов целиком и может отличить нужный документ от похожих соседей там, где статистическое сравнение слов или смысла путается	Отдельный вызов модели — дороже любого из шагов без модели; имеет смысл только тогда, когда список кандидатов уже есть, то есть это шаг поверх уже готового результата — самостоятельным способом искать его не назвать

Структура замера: пятьсот документов, пятьдесят вопросов, три группы

Корпус. 500 документов SberQuAD, включая 8 путающих кластеров — групп документов, которые делят одну и ту же редкую сущность. Пример: группа «Звёздный» — один документ о звездолётах, другой о концертных программах середины 1980-х; оба используют слово «звёздный», но отвечают на совершенно разные вопросы.

ГРУППА	СКОЛЬКО	ПРИМЕР ВОПРОСА	ОЖИДАЕМЫЙ ОТВЕТ	ПОЧЕМУ ЭТА ГРУППА
Лексический разрыв	16	«Когда произошло сражение?»	16 апреля 1746 года	Вопрос и документ почти не делят слов — ожидаемая зона поиска по смыслу
Точное совпадение	16	«Кто в 1395 году принял титул герцога Миланского?»	Джангалеаццо Висконти	Год и титул из вопроса стоят в тексте буквально — ожидаемая зона поиска по словам
Нейтральные	18	«Чем был обмен по бартеру в торговле?»	Единственной доступной формой торговли	Класс выделен по отсутствию общего числа с текстом — степень лексического сходства тут значения не имеет. Группа нейтральная: ни разрыв, ни точное совпадение, хотя внутри неё есть и вопросы, лексически близкие к тексту

Поиск по смыслу считает `text-embedding-3-small` через OpenRouter — та же модель эмбеддинга, что уже работала в Кейсе 1 этого занятия, косинусное сходство со всеми 500 документами, без порога отсечения. Метрика для всех вариантов одна — recall@1 и recall@3 (нашёлся ли верный документ первым или среди первых трёх) и средний ранг верного документа по эталонной разметке SberQuAD. Результатов на этом кадре нет — дальше смотрим, что получилось.

Что показал прогон на 500 документах и 50 вопросах

Лучший результат по первому месту: поиск по смыслу находит верный документ сразу на 1-м месте в 43 вопросах из 50 (86%).

МЕТОД	RECALL@1	RECALL@3	СРЕДНИЙ РАНГ ВЕРНОГО (ИЗ 500)
Искать по словам	0,74	0,80	9,9
Искать по смыслу (вектор)	0,86	0,92	2,0
Искать обоими, объединить (гибрид, RRF)	0,84	0,94	1,6

Свод по тем же трём группам, что назвал прошлый такт (recall@1):

ГРУППА ВОПРОСОВ	СЛОВА	СМЫСЛ	ГИБРИД
Лексический разрыв (16)	0,25	**0,94**	0,69
Точное совпадение (16)	**1,00**	0,81	0,94
Нейтральные (18)	**0,94**	0,83	0,89

Систематическая находка: гибрид не выигрывает НИ В ОДНОЙ из трёх групп по отдельности. Его агрегатный результат (0,84) складывается не из того, что он где-то лучший, а из того, что он нигде не худший.

Что подтвердил замер по каждому из трёх сигналов

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Искать по словам	Совпадение токенов запроса и документа, без модели	Дёшево и быстро — поиск сам ничего не стоит	recall@1 = 0,74 — худший из трёх; если не на первом месте, в среднем на 9,9-й позиции из 500 — лексический разрыв и путающие кластеры сбивают точное совпадение
Искать по смыслу (вектор)	Эмбеддинг запроса сравнивается с эмбеддингами кусков базы	recall@1 = 0,86 — лучший результат среди методов без модели на ранжировании; средний ранг 2,0	recall@3 (0,92) чуть ниже гибрида; слабее там, где важно точное совпадение имени или числа
Искать обоими, объединить (гибрид, RRF)	Два независимых ранжирования сливаются в одно по правилу Reciprocal Rank Fusion	Лучший recall@3 (0,94) и средний ранг (1,6) среди методов без модели на ранжировании	recall@1 = 0,84 — ниже, чем у чистого вектора (0,86): слабый сигнал слов иногда перетягивает объединённый голос не туда, где вектор был прав сразу

Следующий шаг после гибрида: пересмотр моделью

Обе основы считает `text-embedding-3-small` (поиск по смыслу) — реранкер поверх неё добавляет отдельный вызов `gpt-4o-mini`.

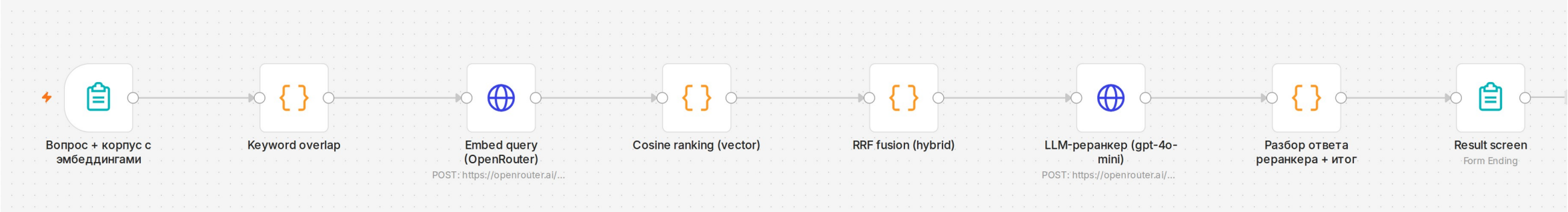
ОСНОВА + РЕРАНКЕР	RECALL@1	$/ВОПРОС (ПОИСК)
Гибрид + LLM-реранкер	**0,96** (было 0,84)	$0,000329
Вектор + LLM-реранкер	**0,96** (было 0,86)	$0,000327

Пример исправления. Вопрос «Какие наиболее популярные произведения были созданы на протяжении 1984–1986 гг» (gold-документ — член путающего кластера): чистый вектор нашёл его на 10-м месте, гибрид — на 2-м; реранкер поднял оба варианта на 1-е.

Поиск — объединение минимум двух сигналов

Поиск — объединение минимум двух разных сигналов релевантности: одного сигнала тут мало, прежде чем решать, какой кусок класть в промт. У одного сигнала (слова) своя область, где он выигрывает — точные совпадения, имена, числа; у другого (смысл) — своя — пересказ своими словами. После объединения сигналов — необязательный дополнительный шаг: отдать финальный порядок на пересмотр модели, если цена отдельного вызова и добавленная задержка оправданы ценой ошибки ранжирования.

Настоящий холст: от вопроса до реранкера



Keyword overlap считает ранжирование по словам первым узлом. Дальше — эмбеддинг вопроса и косинусное ранжирование по смыслу. Только когда оба списка готовы, RRF fusion сливает их — и лишь затем, необязательным шагом, в игру входит LLM-реранкер.

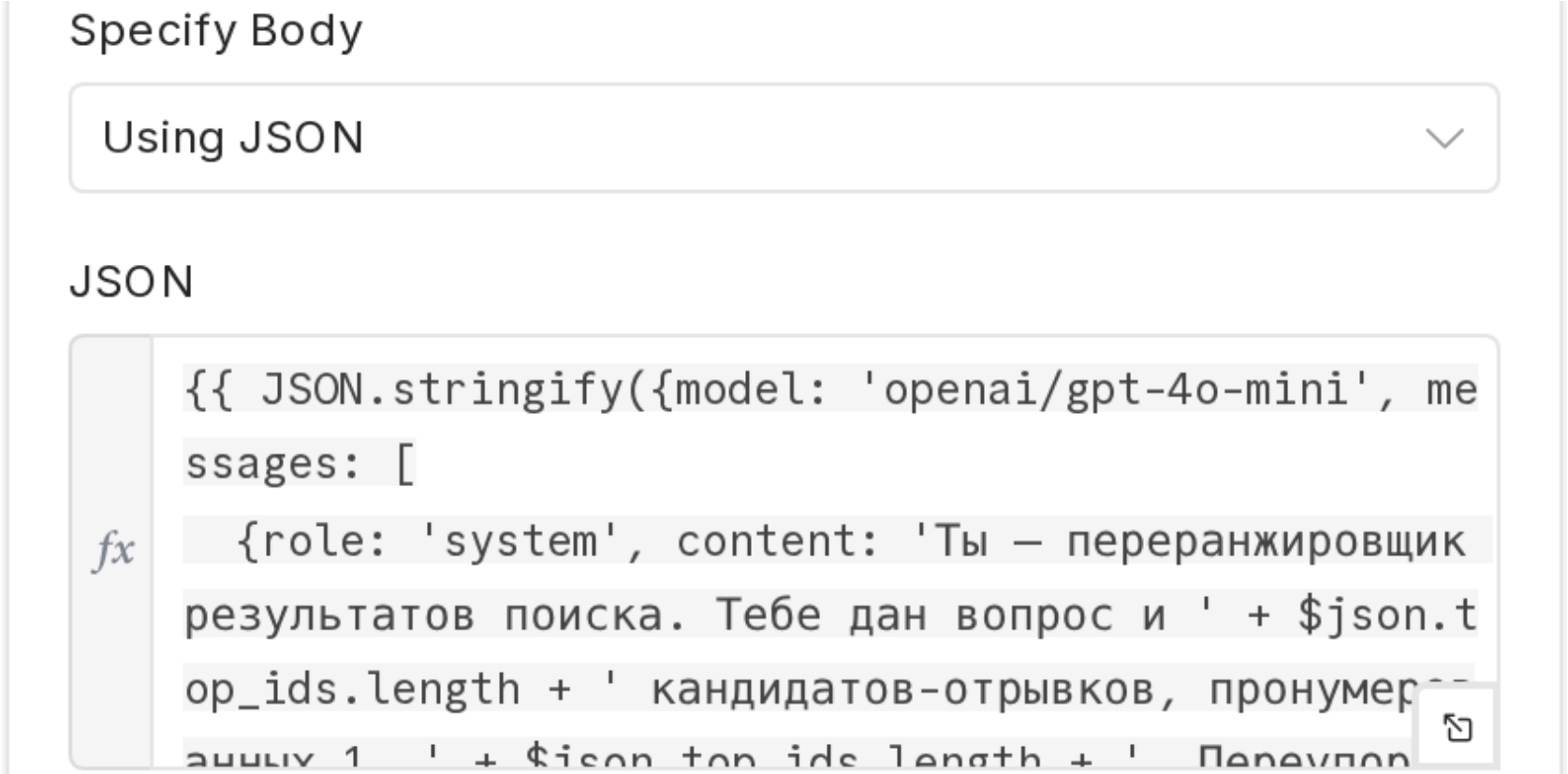
Настоящая панель: код узла слияния

JavaScript

```
1  const d = $input.first().json;
2  function rrf(rankA, rankB, k=60) {
3    const scores = {};
4    [rankA, rankB].forEach(rank => { rank.forEach((id, i) => {
5      scores[id] = (scores[id]||0) + 1/(k+i+1); }); });
6    return Object.keys(scores).sort((a,b)=>scores[b]-scores[a]);
7  }
8  const hybrid_rank = rrf(d.keyword_rank, d.vector_rank, 60);
9  const TOP_N = Math.min(10, hybrid_rank.length);
10 const topIds = hybrid_rank.slice(0, TOP_N);
11 const byId = {}; d.corpus.forEach(c=>byId[c.id]=c.text);
12 const candidatesText = topIds.map((id,i)=>`${i+1}.
   ${byId[id]}`).join('\n\n');
13 return [{json: {...d, hybrid_rank, top_ids: topIds, candidates_text:
   candidatesText}}];
```

Строка 8 — вся идея узла: `rrf(d.keyword_rank, d.vector_rank, 60)`. Два уже готовых списка складываются в один по правилу «чем выше документ в обоих списках, тем выше в объединённом».

Настоящая панель: тело запроса реранкера



> *«Ты — переранжировщик результатов поиска. Тебе дан вопрос и 10 кандидатов-отрывков, пронумерованных 1..10. Переупорядочи номера кандидатов от самого релевантного вопросу к наименее релевантному. Ответь СТРОГО JSON-массивом чисел...»*

Что видно Ольге, а что — обратившемуся

Обратившемуся гибридный поиск виден как более стабильный ответ независимо от того, как именно сформулирован вопрос — точным термином или пересказом своими словами. Ольге видно число: доля вопросов, на которые процесс нашёл верный документ среди первых результатов, выше у гибрида, чем у любого из двух способов по отдельности.

Принцип переносится на любой оркестратор: поиск по смыслу не заменяет поиск по словам и не наоборот — у каждого свой класс вопросов, на которых он сильнее, и объединение обоих результатов не требует заранее знать, какой класс вопроса придёт следующим.

Когда гибрид не нужен и когда реранкер подводит

Критерий. Гибридный поиск — кандидат на снятие внутри Кейса 3 (пункт №2 общего порядка сокращения, после снятия детального разбора perceptual hash в Кейсе 4), если слот не вмещает весь материал: метод работает исправно, но на небольшой однородной базе он не нужен настолько, насколько нужен на большой и разнородной.

КОГДА НУЖНО

База большая, вопросы формулируются и точными терминами, и пересказом своими словами

Цена ошибки ранжирования высока — обратившийся получит неверный ответ по второму-третьему кандидату

КОГДА НЕ НУЖНО

База маленькая и однородная по формулировкам вопросов

Разница между одним способом и объединением двух статистически неразличима на этой базе

Реранкер — вторая граница того же рода, с двумя сторонами сразу. Он добавляет 10–12 процентных пунктов к recall@1 — но стоит отдельного вызова модели ($0,00033, дороже всего пайплайна Кейса 1) и задержки (+1,6–1,9с). Он может поднять gold-документ с 10-го места на 1-е — так произошло с вопросом про популярные произведения середины 1980-х. И он же способен столкнуть уже верный результат с 1-го места на 3-е — так случилось с вопросом «Пример водных преград?», где чистый вектор и так нашёл верный документ первым. Если документа нет даже среди top-10 кандидатов до реранка — как в вопросе про бартерный обмен в торговле, где вектор нашёл gold только на 26-м месте, — реранкер его не поднимет вовсе: он переставляет только то, что ему показали, весь пул документов заново он не трогает.

Третья строка оси занятия закрыта: поиск по смыслу находит документ на первом месте чаще, чем поиск по словам, на жёстком корпусе — но не гарантированно и не универсально; это зависит от корпуса и вопросов, а объединение сигналов не побеждает по каждой метрике сразу.

Кейс 4. Поиск по изображениям

Позапрошлое занятие показало пределы передачи картинки модели. Прошлое — что модель умеет её прочитать. Сегодня — та же задача хранения и поиска, что в трёх предыдущих кейсах, только для сканов и фотографий.

Хранение базы знаний | Разбиение документа на части | Чем искать нужный кусок | **Поиск по изображениям**

Как найти прошлый похожий скан



Потребность. Учебный центр получает сканы бланков и чеков в заявках регулярно. Нужно найти среди прошлых заявок похожий скан: прочитать текущий тут мало.

Версия 1 — естественная первая попытка, прямое продолжение трёх предыдущих кейсов: подписать картинку текстом (распознать текст или получить текстовое описание от модели) и искать эту подпись тем же механизмом, что в Кейсе 3 — как обычный текстовый документ.

Один из трёх проверенных сегодня вариантов закончится честно показанным провалом — настоящей неудачей с выученным уроком; ослабленной версией успеха её не назвать.

Как искать похожий скан среди прошлых

Как искать похожий скан среди прошлых?

подписать картинку словами и искать как текст

искать картинку по картинке напрямую

искать точные повторы отдельным быстрым способом

Что делает каждое из трёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Подписать и искать как текст	Текст чека превращается в тот же текстовый эмбеддинг, что в Кейсах 1-3	Работает корректно: эмбеддинги содержательно различаются между чеками, переиспользует весь механизм Кейсов 1-3	Подпись — пересказ картинки словами: расположение полей, визуальный вид бланка, печать, рукописная пометка теряются до того, как дошли до поиска
Искать картинку по картинке (мультимодальный эмбеддинг)	Модель превращает саму картинку в числовое представление без промежуточного текста	Теоретически не теряет ничего из того, что отбрасывает подпись	**На этом стенде не работает на реальном разрешении скана — разбор следующим тактом**
Искать точные повторы (perceptual hash)	Короткий числовой отпечаток картинки, вычисляемый локально, без модели	Надёжно ловит почти-дубли при пересжатии — без единого вызова модели	Решает только подзадачу «тот же самый скан, слегка пересжатый»; задачу «похожий по смыслу документ» этим не закрыть

Провал, часть 1: модели плана не существует

Шаг 1. `voyage-multimodal-3`, модель, на которую план рассчитывал изначально, не найдена на OpenRouter этого стенда ни под одним из проверенных написаний id. Полный каталог моделей (452 позиции) не содержит ни одной модели с «voyage» или «embed» в названии вообще.

Перебор реальных кандидатов нашёл ровно одну рабочую мультимодальную модель — `amazon/titan-embed-image-v1` — она принимает и текст, и картинку, и отдаёт вектор без ошибки.

Структура замера: одна пара чеков, три разрешения

Вызов. `POST https://openrouter.ai/api/v1/embeddings`, `model: amazon/titan-embed-image-v1`, вход — одна строка `data URL` (`…`), без отдельного текстового поля.

РАЗРЕШЕНИЕ	JPEG КАЧЕСТВО	ПРИМЕРНЫЙ РАЗМЕР BASE64
300×450 (реальный скан — тот размер, с которым работают остальные кейсы занятия)	q80	~18 000–26 000 символов
128×192 (уменьшенное превью)	q75	~4 300–4 600 символов
64×96 (телефонная иконка-превью)	q80	~6 000–8 000 символов

Ожидание, если модель действительно видит картинку: простого ответа без ошибки тут не хватит. Два РАЗНЫХ чека на одном и том же разрешении обязаны дать два РАЗНЫХ вектора — косинусное расстояние между ними должно отличаться от 1,0. Проверка «пришёл ли ответ нужной формы» этого не показывает: она проходит одинаково и тогда, когда модель честно использует картинку, и тогда, когда она её игнорирует.

Свод и разбор: ни одно разрешение не дало надёжного различения

Свод. На разрешении, которое реально нужно по задаче, — провал. Модель не использует картинку ни на одном из двух проверенных разрешений настоящего скана (300×450, 128×192) — различие появляется только на самом маленьком проверенном размере, и там оно статистически неотличимо от шума.

РАЗРЕШЕНИЕ	ВЕРДИКТ	ЧТО ПОКАЗАЛ ТЕСТ
300×450 (реальный скан)	**ПРОВАЛ — ОДИНАКОВЫЙ ВЕКТОР**	cosine = 1,0 до 9 знаков для любых двух разных чеков
128×192	**ПРОВАЛ — ОДИНАКОВЫЙ ВЕКТОР**	cosine = 1,0 — подтверждено повторной живой проверкой (2026-11-02)
64×96 (маленькое превью)	**ФОРМАЛЬНО РАЗЛИЧАЕТ, НО ≈ ШУМ**	корреляция визуального сходства с текстовым по всем парам −0,077 — статистически неотличимо от нуля

Ответ без ошибки и обработанный вход — разные вещи. Единственный способ это проверить — подать на вход две заведомо разные картинки и сравнить, различается ли выход; тест «пришёл ли ответ правильной формы» этого не покажет.

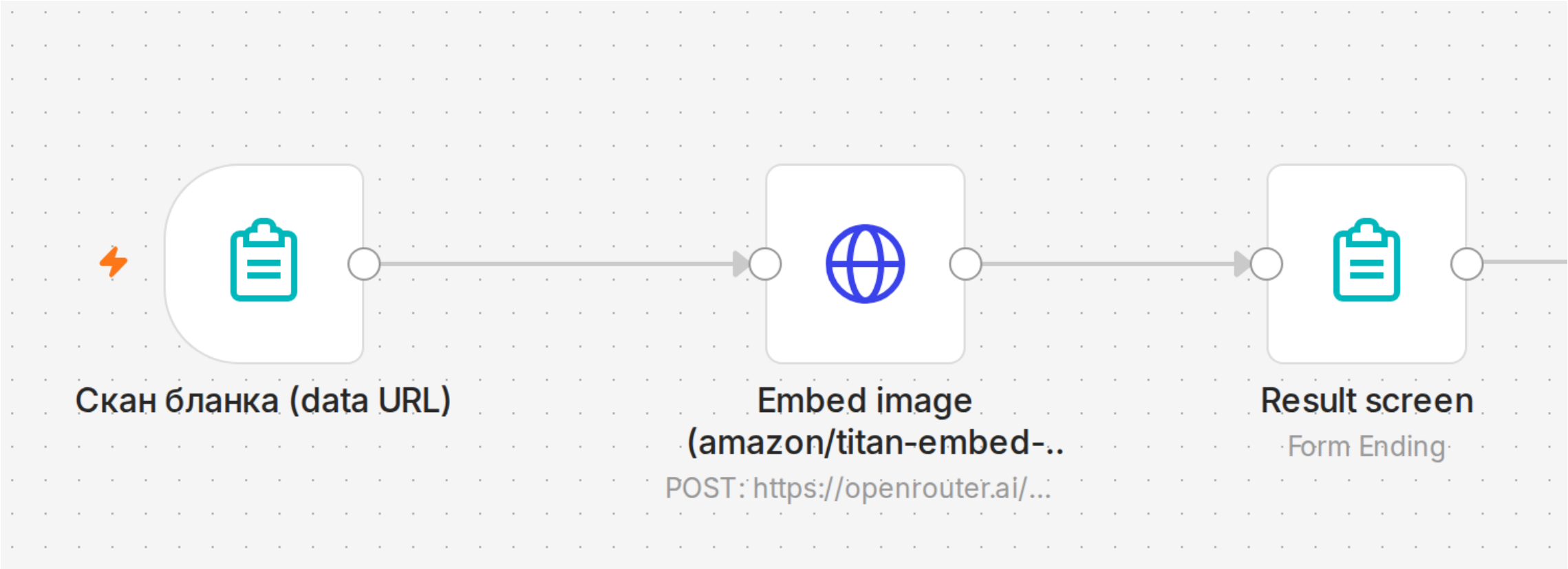
Для сравнения, другой единицей измерения. Perceptual hash (классика, без обращения к модели) на этих же чеках отличает дубль от случайного чужого уверенно — расстояние 0–1 бит Хэмминга до дубля против 14–49 бит до случайного чужого чека. Это не строка этой таблицы: биты Хэмминга и косинус — разные единицы (подробный разбор границы самого phash — в такте границ этого кейса, дальше).

Правильная идея без готовой реализации

Правильная архитектура — сравнивать картинки в их собственном числовом представлении: текстового посредника тут недостаточно. Это архитектурный тезис, и он остаётся верным независимо от того, какая конкретная модель его реализует.

Рабочее решение задачи на сегодняшнем стенде складывается из двух оставшихся частей разбора: подписать и искать как текст — для смыслового поиска «похожий документ по содержанию» — и perceptual hash отдельным быстрым шагом — для задачи «точно такой же скан, просто пересжат». Это то же правило, что уже встретилось в Кейсе 3; компромиссом вместо мультимодального эмбеддинга его не назвать: один сигнал не покрывает все случаи.

Настоящий холст: попытка, которая не удалась



Холст на стенде реально собран и реально выполнен — но архитектурный тезис слоя 1 этим прогоном не подтверждён: рабочей демонстрации «поиск картинки по картинке» на этом стенде нет, только честно показанный и воспроизведённый провал.

Настоящая панель: тело запроса — выражение, без текста



`Body Content Type: JSON` → `Specify Body: Using JSON` → `JSON: {{ $json.body }}` — то же устройство тела запроса, что уже показывал узел эмбеддинга текста в первом кейсе этого занятия, только здесь внутри `$json.body` лежит картинка, без текста.

Что видно Ольге, а что — обратившемуся

Обратившемуся не видно, как именно процесс нашёл прошлый похожий бланк — видно, что он его нашёл, когда это возможно по содержанию, и что система отдельно и надёжно отличает буквальный повторный скан той же заявки от нового документа. Ольге видно границу: для какого класса похожести (смысловая — «такой же тип бланка») нужен один механизм, а для какого (буквальная — «тот же самый документ, просто переснятый») — другой, и дешевле.

Принцип, переносимый на любой оркестратор: архитектурная идея «сравнивать данные в их собственном пространстве, минуя пересказ» может быть верной и при этом не иметь готовой рабочей реализации на конкретном провайдере в конкретный момент — проверка этого занимает один контролируемый эксперимент с двумя разными входами, и пропустить этот эксперимент значит унести нерабочее решение, которое выглядело рабочим на первом тесте.

Где метод точных повторов перестаёт быть надёжным

ЧЕК	МЯГКИЙ ДУБЛЬ (БИТ ИЗ 64)	ЖЁСТКИЙ ДУБЛЬ	СЛУЧАЙНЫЙ ЧУЖОЙ
receipt_07	1	27	14
receipt_18	0	10	24
receipt_17	0	30	35
receipt_04	1	40	23
receipt_11	0	10	44
receipt_19	0	10	49

Граница применимости. Под поворотом расстояние растёт до 10–40 бит — в половине чеков сопоставимо со случайным чужим. Метод решает подкласс «тот же скан, пересжатие»; поворот в этот подкласс не входит.

Когда стоит пробовать поиск картинки по картинке напрямую — и когда точно нет на этом стенде. Стоит пробовать: когда провайдер и модель подтверждают приём картинки отдельным контролируемым тестом на двух заведомо разных входах, на том самом разрешении, которое нужно по задаче; одной беглой проверки «пришёл ли ответ» тут недостаточно. Не стоит прямо сейчас, на этом стенде: `amazon/titan-embed-image-v1` через OpenRouter на разрешении реального скана (300×450) — подтверждённый контролируемым тестом провал; предположением его не назвать, и повода избегать мультимодальных эмбеддингов вообще это тоже не даёт.

Четвёртая строка оси занятия закрыта: если умеем искать текст, картинку тем же способом не найдём: подпись теряет видимое, прямое сравнение картинок может ответить без ошибки и при этом не использовать вход вовсе.

Релевантно ≠ верно

Четыре кейса сегодня учили находить нужный кусок базы знаний и класть его в промт перед ответом. Всё это время за кадром стоял не проверенный явно допуск: раз система нашла кусок, топически относящийся к вопросу, значит ответ на его основе окажется верным.

Случай 1. Google AI Overviews и совет про пиццу

В 2024 году функция обзоров с помощью ИИ в поиске Google несколько раз выдавала пользователям советы, взятые из источников, которые по форме звучали как достоверные, а по содержанию были шуткой: один из широко обсуждавшихся случаев — рекомендация добавлять нетоксичный клей в соус для пиццы, источником которой оказался старый шуточный комментарий на интернет-форуме.

Система нашла текст, который действительно был об этом — про сыр, про пиццу, про то, как сделать её менее текучей. По меркам поиска — успех: релевантность высокая. По меркам ответа пользователю — провал: источник оказался шуткой — кулинарной рекомендацией он не был.

Случай 2. Чат-бот городской администрации Нью-Йорка

Отдельный, документированный журналистским расследованием случай 2024 года: городской чат-бот Нью-Йорка, созданный в помощь владельцам малого бизнеса, на прямые вопросы о городских правилах несколько раз выдавал уверенные, грамматически безупречные ответы, которые были юридически неверны или напрямую противоречили закону города — например, бот говорил, что можно забирать себе чаевые сотрудников, и что можно отказывать арендаторам с жилищными субсидиями.

Оба случая технически сработали правильно: нашли релевантный по теме текст и построили на его основе грамматически корректный ответ. Провал сидит в допущении, что тематическая близость равна фактической верности — механика поиска тут ни при чём.

Почему это касается всех четырёх кейсов сегодня

Ни Кейс 1 (хранение), ни Кейс 2 (резка), ни Кейс 3 (поиск), ни Кейс 4 (поиск по изображениям) не застрахованы от этого автоматически самим фактом, что они используют поиск вместо того, чтобы передавать модели весь объём данных целиком. Поиск, который правильно нашёл наиболее похожий по смыслу фрагмент — ровно то, что измеряли метрики recall и ранг во всех четырёх кейсах, — отвечает только на вопрос «это похоже на то, что спросили» — вопрос «это верно» он не затрагивает.

Это не повод отказаться от поиска по базе знаний — отказ вернул бы занятие к промт-стаффингу, который Кейс 1 уже показал невыгодным при росте базы. Это повод для отдельного шага, который следующие занятия этой дуги строят отдельно: оценка качества и выключатель для ответа, построенного на найденном материале.

Четыре строки оси: чем оказалось

ВЫГЛЯДИТ НАДЁЖНЫМ	ЧЕМ ОКАЗЫВАЕТСЯ
если база выросла — просто перечислим её всю в промте, как на C9	Accuracy держится на одном уровне (~0,7 на 10/50/200) без предупреждающего сигнала, стоимость при этом растёт ×17,0 — а на 500 документах запрос физически отклонён (HTTP 400, контекст превышен); keyword держит recall@1=0,8 на всех размерах (слабость метода — рост базы тут ни при чём), vector — 1,0 на 10/50/200 и 0,9 на 500, ~$0,00005 на любом размере
порезать документ на части можно как угодно, главное чтобы кусок попал в хранилище	Резка по символам рвёт ответ в 2,8–12,5% случаев; резка по предложениям — почти всегда 0–1,4%; «удобный на вид» размер не гарантирует качества без проверки на своих данных
поиск по смыслу (вектор) и так находит то же самое, что поиск по словам, только умнее	На жёстком корпусе вектор обгоняет слова по recall@1 (0,86 vs 0,74) — но объединение сигналов не побеждает по каждой метрике разом; реранкер поднимает recall@1 до 0,96 ценой отдельного вызова модели и собственных ошибок
если умеем искать текст — картинку найдём тем же способом, просто подписав её словами	Подпись теряет то, что видно на картинке; прямое сравнение картинок на этом стенде технически не работает — модель отвечает без ошибки, но возвращает одинаковый вектор для разных изображений

Откуда процесс берёт факты и почему считает их верными?

Счёт занятия: что измерено, что осталось непроверенным

Четыре кейса, четыре самостоятельных замера на реальных данных — SberQuAD для текста (Кейсы 1-3), CORD для изображений (Кейс 4). Ни один — на придуманном числе. Один из четырёх прогонов — мультимодальный эмбеддинг в Кейсе 4 — закончился честно показанным нерабочим результатом — доработанным суррогатом его не выдали.

Хранилище (Кейс 1) экономит на стоимости ответа, но добавляет шаг подготовки данных и риск устаревшего индекса — копия базы не обновляется сама, когда меняется источник. Резка по смыслу (Кейс 2) точнее резки по счёту символов, но чуть сложнее в реализации и зависит от качества разбивки текста на предложения. Гибридный поиск (Кейс 3) надёжнее одного способа, но стоит лишнего шага вычисления и слияния на каждый запрос — и не оправдан на маленькой однородной базе; реранкер добавляет ещё один платный шаг сверху. Поиск по изображениям (Кейс 4) там, где он действительно нужен, требует отдельно проверенного компонента под каждый класс похожести — и ни один удобный универсальный ответ не оказался готов к использованию без собственной проверки.

Что дальше

Может ли процесс не только отвечать, но и действовать от имени компании?

Вопрос, с которого начался прошлый семинар («может ли процесс отвечать обратившемуся сам»), и вопрос, с которого начался сегодняшний («откуда процесс берёт факты»), вместе подводят к следующему шагу ступенчатой дуги курса: что происходит, когда процессу дают не только знание, но и полномочие — право не просто предложить ответ, но и совершить действие от имени компании.

И то, чего сегодняшнее занятие не чинит: разбавитель этого дня — «найдено» ≠ «верно» — сам не закрывается ни выносом базы наружу, ни правом действовать. Это отдельная ступень: оценка качества и выключатель, дальше по дуге.